Understanding CrashLoopBackOff in Kubernetes

📘 Overview

In Kubernetes, a **CrashLoopBackOff** error indicates that a container inside a Pod has **crashed repeatedly** after starting.
Kubernetes automatically tries to restart the container, but if it fails multiple times, it enters a **CrashLoopBackOff** state — meaning it's waiting for a certain amount of time before retrying again (using exponential backoff).

Simply put:

> The container starts → crashes → restarts → crashes again → and Kubernetes backs off for longer and longer intervals.

🔹 What Causes CrashLoopBackOff?

CrashLoopBackOff can occur due to various reasons. Below are the most common causes:

Category	Common Issue	Description
Application Errors	App crashes on startup	The main process inside the container terminates due to code issues, bad configuration, or missing dependencies.
Configuration Errors	Invalid environment variables or arguments	The app fails to start because required configuration values are missing or incorrect.
Probes Failing	Readiness, Liveness, or Startup probes misconfigured	The app takes longer to start than the probe allows, causing K8s to restart it.
Resource Limits Exceeded	OOMKilled (Out of Memory)	The container consumes more memory/CPU than the defined limits.
Secrets or ConfigMaps Missing	Missing or incorrectly mounted secrets/configs	The app cannot find the required file or variable to start.
Init Container Failure	Failure in the init container	The init process fails, blocking the main container from starting.
Image or Command Issues	Wrong entrypoint, command, or image tag	The container fails immediately upon execution.

🔹 How to Diagnose the Root Cause

Follow these steps to identify and troubleshoot the cause of the CrashLoopBackOff:

1️⃣ Check Pod Status

```
kubectl get pods
```

Look for Pods showing the **CrashLoopBackOff** status.

2️⃣ Describe the Pod

```
kubectl describe pod <pod-name>
```

- Scroll to the **Events** section at the bottom.
- Look for messages like:
    - “Back-off restarting failed container”
    - “OOMKilled”
    - “Liveness probe failed”
    - “CrashLoopBackOff”

These clues usually indicate what went wrong.

3️⃣ View Container Logs

```
kubectl logs <pod-name> --previous
```

> Use `--previous` to see logs from the **last crashed container instance**.

Check for:

- Application errors
- Missing environment variables
- Dependency issues
- Startup exceptions

4️⃣ Inspect Probes (if configured)

```
kubectl get pod <pod-name> -o yaml
```

Verify if your probes are properly configured:

- **livenessProbe**
- **readinessProbe**
- **startupProbe**

If your application takes time to start, you may need to increase:

```
initialDelaySeconds: 30
```

```
periodSeconds: 10
```

5️⃣ Check Resource Limits

```
kubectl describe pod <pod-name>
```

If you see:

```
State:   Terminated
Reason:  OOMKilled
```

→ The container ran out of memory.

Fix: Increase resource limits in your deployment YAML:

```
resources:
  requests:
    memory: "512Mi"
  limits:
    memory: "1Gi"
```

🔹 Common Fixes

Problem	Possible Solution
Application crashes on start	Fix the startup script or application logic.
Missing environment variables	Add them using ConfigMap or Secret references.
Liveness probe fails	Increase probe delay or period.
OOMKilled	Increase memory/CPU limits.
Image or command issue	Verify the Docker image, entrypoint, and command fields.
Missing secrets/config	Ensure ConfigMaps and Secrets exist and are correctly mounted.

🔹 Example Scenario

Issue:

Application fails to connect to a database and crashes.

Log Output:

```
Error: DATABASE_URL not set
```

Root Cause:

Missing environment variable `DATABASE_URL`.

Fix:

Update Deployment YAML:

```
env:
  - name: DATABASE_URL
    valueFrom:
      secretKeyRef:
        name: db-secret
        key: db-url
```

Apply the fix:

```
kubectl apply -f deployment.yaml
```

🔹 Debugging Tip

If you want to **prevent automatic restarts** to debug manually:

```
kubectl edit deployment <deployment-name>
```

Change:

```
restartPolicy: Never
```

Now you can `exec` into the container and debug interactively.

🔹 Summary

Concept	Description
CrashLoopBackOff	Container repeatedly crashes and restarts.
Root Cause	Application errors, misconfigurations, missing resources, or probe failures.
Troubleshooting Commands	`kubectl describe pod`, `kubectl logs --previous`, `kubectl get pod -o yaml`.
Prevention	Proper configuration, resource allocation, and health probe tuning.

1. ImagePullBackOff

📘 What It Means

`ImagePullBackOff` occurs when Kubernetes is **unable to pull the container image** from the container registry (like Docker Hub, ECR, or GCR).
It goes into a *BackOff* state, meaning it keeps retrying after increasing time intervals.

🔍 Common Causes

Cause	Description
❌ **Incorrect Image Name or Tag**	The specified image name or tag doesn't exist in the registry.
🔐 **Private Registry Authentication Failure**	Kubernetes can't access a private registry due to missing or incorrect credentials.
🌐 **Network Issues**	Cluster nodes can't reach the container registry (firewall or DNS issue).
🧱 **Image Not Yet Pushed**	The image hasn't been built or pushed to the registry.
🗒️ **Rate Limiting (Docker Hub)**	Docker Hub limits anonymous image pulls.

🧰 How to Troubleshoot

1️⃣ Check Pod Status

```
kubectl get pods
```

2️⃣ Describe the Pod

```
kubectl describe pod <pod-name>
```

➡️ In the **Events** section, you'll see something like:

```
Failed to pull image "myapp:v2": image not found
Back-off pulling image "myapp:v2"
```

3️⃣ Verify the Image

Try pulling manually from the node:

```
docker pull <image-name>:<tag>
```

4️⃣ Check Image Pull Secrets

If it's a private registry:

```
kubectl get secrets
```

Ensure your Deployment/Pod spec includes:

```
imagePullSecrets:
  - name: myregistry-secret
```

🧩 How to Fix

✅ Verify the image name and tag
✅ Push the image to the registry
✅ Add correct **imagePullSecrets** for private registries
✅ Fix network/DNS issues
✅ Avoid Docker Hub rate limits by using authenticated pulls or caching

---

🧮 2. OOMKilled (Out of Memory Killed)

📘 What It Means

`OOMKilled` means your container **exceeded its memory limit**, and the Kubernetes **OOM (Out of Memory) Killer** terminated it to protect the node.

🔍 Common Causes

Cause	Description
🧠 **Memory Limits Too Low**	The Pod's memory limit is lower than what the app needs.
🐞 **Memory Leak**	The application keeps consuming memory without releasing it.
⚙️ **High Workload**	Unexpected spikes in load cause memory usage to rise.
🧮 **No Limit Defined**	The container uses excessive memory until the node kills it.

🧰 How to Troubleshoot

1️⃣ Describe the Pod

```
kubectl describe pod <pod-name>
```

Look for:

```
State:   Terminated
Reason:  OOMKilled
```

2️⃣ Check Resource Usage

If using metrics server:

```
kubectl top pod <pod-name>
```

Or check monitoring dashboards (Prometheus, Grafana, CloudWatch).

3️⃣ Analyze Application Logs

```
kubectl logs <pod-name> --previous
```

See if the application was using too much memory before termination.

🧩 How to Fix

✅ Increase memory limits in your YAML file:

```
resources:
  requests:
    memory: "512Mi"
  limits:
    memory: "1Gi"
```

✅ Optimize your application to reduce memory usage.
✅ Use autoscaling (HPA or VPA) for high-load scenarios.
✅ Continuously monitor memory metrics using Prometheus or CloudWatch.

💡 Tip:

If your app needs time to warm up and load data, ensure limits aren't too strict — start with generous limits and tune them over time.

🌐 3. Insufficient IP Addresses

📘 What It Means

`Insufficient IP Addresses` error occurs when Kubernetes **can't assign a new IP address to a Pod**, typically due to **IP exhaustion** within the node's or cluster's network range.

🔍 Common Causes

Cause	Description
🧱 **Subnet Too Small**	The Pod network CIDR (IP range) doesn't have enough IPs for all pods.
🧩 **Too Many Pods per Node**	Node runs out of allocatable Pod IPs.
🌐 **CNI Plugin Misconfiguration**	The networking plugin (e.g., Calico, Cilium, Flannel) has incorrect IP pool settings.
⚙️ **IP Range Overlap**	IP ranges for multiple clusters or networks overlap, causing conflicts.

🧰 How to Troubleshoot

1️⃣ Check Pod Events

```
kubectl describe pod <pod-name>
```

Look for:

```
Failed to create pod sandbox: failed to allocate IP address
```

2️⃣ Check Node Status

```
kubectl get nodes
kubectl describe node <node-name>
```

See if nodes are running out of available pod IPs.

3️⃣ Verify Cluster Network Config

Check the Pod network CIDR used during cluster setup:

```
kubectl cluster-info dump | grep -m 1 cluster-cidr
```

4️⃣ Check CNI Plugin Configuration

For example, with Calico:

```
kubectl get ippools.crd.projectcalico.org
```

🧩 How to Fix

✅ **Increase Pod CIDR range** during cluster setup or CNI configuration.
✅ **Add more nodes** to distribute pods and IP allocations.
✅ **Clean up unused pods or namespaces** to free IPs.
✅ **Reconfigure CNI plugin** (e.g., Calico IP pools) for larger address space.
✅ Avoid overlapping IP ranges between clusters or VPCs.

🧠 Summary

Error	Meaning	Root Cause	Fix
ImagePullBackOff	K8s can't pull image from registry	Wrong image, missing tag, private repo issue, network problem	Verify image, fix registry access, check secrets
OOMKilled	Container used more memory than allowed	Low memory limit or memory leak	Increase memory limits, optimize app
Insufficient IP Addresses	Cluster can't assign new Pod IP	Pod network CIDR exhausted or misconfigured	Expand CIDR, add nodes, fix CNI plugin

Jenkins CI/CD pipeline

Prerequisites (what must be installed / configured)

1. **Jenkins** with these plugins:
    - Pipeline
    - Git
    - SonarQube Scanner for Jenkins
    - JFrog CLI or JFrog Artifactory Plugin (we'll use `jfrog` CLI)

    - Docker Pipeline
    - Kubernetes CLI Plugin or `kubectl` available on agent
    - Credentials Binding
2. **Jenkins global tools** configured (JDK, Maven, Docker if using docker-in-docker agent).
3. **SonarQube** server reachable and configured in Jenkins (with a `sonar` server entry or token in credentials).
4. **Artifactory** account + repository and `jfrog` CLI credentials.
5. **Docker registry** credentials (could be Artifactory Docker repo, Docker Hub, ECR, ACR).
6. **Kubernetes cluster** and `kubeconfig` accessible from Jenkins (store kubeconfig in Jenkins Credentials or mount on agent).
7. (Optional) **Prometheus Operator** in the cluster if you plan to use `ServiceMonitor`. If not present, the remediation uses **pod annotations** and a Prometheus instance configured to scrape pods.
8. Jenkins credentials stored (IDs referenced in the pipeline):
    - `GIT_CREDENTIALS` — SSH key or user/pass for Git
    - `SONAR_TOKEN` — SonarQube token (secret text)
    - `ARTIFACTORY_USER` / `ARTIFACTORY_PASSWORD` — or one credential `ARTIFACTORY_CRED`
    - `DOCKER_REGISTRY_CRED` — Docker registry username/password (or token)
    - `KUBECONFIG` — kubeconfig (Secret file credential) or use a `KUBE_TOKEN` + cluster info

---

Project layout (example)

```
my-app/
├─ src/
├─ pom.xml
├─ Dockerfile
├─ k8s/
│  ├─ namespace.yaml
│  ├─ deployment.yaml
│  ├─ service.yaml
│  ├─ ingress.yaml        (optional)
│  ├─ servicemonitor.yaml (optional - Prometheus Operator)
├─ Jenkinsfile
└─ README.md
```

---

1) `Jenkinsfile` (Declarative Pipeline)

> Copy this `Jenkinsfile` to your repo root. Adjust credential IDs and endpoints to your environment.

```
pipeline {
  agent any

  environment {
    // Replace these with your Jenkins credentials IDs and values
    GIT_CREDENTIALS = 'GIT_CREDENTIALS'            // (optional) Git credential
id
    SONAR_TOKEN      = credentials('SONAR_TOKEN')  // SonarQube token (secret
text)
    ART_USER         = credentials('ARTIFACTORY_USER') // or
ARTIFACTORY_CRED_USER
    ART_PASSWORD     = credentials('ARTIFACTORY_PASSWORD')
    ART_URL          = 'https://artifactory.example.com/artifactory' // change
    ART_REPO         = 'maven-local' // repository name in Artifactory
    DOCKER_REGISTRY  = 'docker.example.com/myrepo' // change to registry/repo
    DOCKER_CRED      = 'DOCKER_REGISTRY_CRED' // Jenkins credentials ID
(username/password)
    IMAGE_TAG        = "${env.BUILD_NUMBER}"
    KUBE_CONFIG_CRED = 'KUBECONFIG' // kubeconfig file credential id
    SONAR_PROJECT_KEY = 'my-app'
  }

  options {
    timestamps()
    buildDiscarder(logRotator(numToKeepStr: '30'))
    ansiColor('xterm')
  }

  stages {

    stage('Checkout') {
      steps {
        checkout([$class: 'GitSCM',
          branches: [[name: '*/main']],
          userRemoteConfigs: [[url: 'git@github.com:youruser/your-repo.git',
credentialsId: env.GIT_CREDENTIALS]]
        ])
      }
    }

    stage('Build') {
      agent { label 'maven' } // optional: a node with Maven installed
      steps {
        sh 'mvn -B clean package -DskipTests=true'
        stash includes: 'target/*.jar', name: 'app-jar'
      }
    }

    stage('Unit Tests') {
      agent { label 'maven' }
      steps {
        sh 'mvn -B test'
        junit 'target/surefire-reports/*.xml'
      }
    }

    stage('SonarQube Analysis') {
```

```
      agent { label 'maven' }
      steps {
        withCredentials([string(credentialsId: 'SONAR_TOKEN', variable:
'SONAR_TOKEN')]) {
          // If you have SonarQube Jenkins plugin configured you can use
withSonarQubeEnv
          sh """
            mvn -B sonar:sonar \
              -Dsonar.projectKey=${SONAR_PROJECT_KEY} \
              -Dsonar.host.url=https://sonarqube.example.com \
              -Dsonar.login=${SONAR_TOKEN}
          """
        }
      }
    }

    stage('Publish Artifact to Artifactory') {
      agent { label 'jnlp' }
      steps {
        unstash 'app-jar'
        // Use jfrog CLI or curl to deploy. Example using jfrog CLI (ensure
jfrog CLI is installed on agent).
        withCredentials([usernamePassword(credentialsId: 'ARTIFACTORY_CRED',
usernameVariable: 'ART_USER', passwordVariable: 'ART_PASSWORD')]) {
          sh '''
            # configure jfrog CLI
            jfrog rt c --url=${ART_URL} --user=${ART_USER} --password=$
{ART_PASSWORD} art-conn --interactive=false
            # upload jar - adjust target path as per your repo layout
            jfrog rt u "target/*.jar" ${ART_REPO}/my-app/${env.BUILD_NUMBER}/ --
flat=true --build-name=my-app --build-number=${BUILD_NUMBER}
            jfrog rt bce my-app ${BUILD_NUMBER}
            jfrog rt bp my-app ${BUILD_NUMBER}
          '''
        }
      }
    }

    stage('Build & Push Docker Image') {
      agent { label 'docker' } // node having docker client
      steps {
        unstash 'app-jar'
        script {
          // build image
          sh """
            cp target/*.jar app.jar || true
            cat > Dockerfile <<'DOCK'
            FROM eclipse-temurin:17-jre
            ARG JAR_FILE=app.jar
            COPY ${JAR_FILE} /app/app.jar
            ENTRYPOINT ["java","-jar","/app/app.jar"]
            EXPOSE 8080
            DOCK
          """
          docker.withRegistry("https://${DOCKER_REGISTRY}", "${DOCKER_CRED}") {
            def image = docker.build("${DOCKER_REGISTRY}:$IMAGE_TAG")
            image.push()
            // tag latest if desired
            image.push("latest")
          }
        }
      }
    }
```

```
    stage('Deploy to Kubernetes') {
      agent { label 'kubectl' } // node with kubectl installed
      steps {
        withCredentials([file(credentialsId: env.KUBE_CONFIG_CRED, variable:
'KUBECONFIG_FILE')]) {
          sh '''
            export KUBECONFIG=${KUBECONFIG_FILE}
            # replace image tag in k8s manifest and apply
            sed "s|DOCKER_IMAGE_PLACEHOLDER|${DOCKER_REGISTRY}:$IMAGE_TAG|g"
k8s/deployment.yaml > k8s/deployment-${BUILD_NUMBER}.yaml
            kubectl apply -f k8s/namespace.yaml || true
            kubectl apply -f k8s/service.yaml
            kubectl apply -f k8s/deployment-${BUILD_NUMBER}.yaml
          '''
        }
      }
    }

    stage('Verify Deployment') {
      agent { label 'kubectl' }
      steps {
        withCredentials([file(credentialsId: env.KUBE_CONFIG_CRED, variable:
'KUBECONFIG_FILE')]) {
          sh '''
            export KUBECONFIG=${KUBECONFIG_FILE}
            kubectl rollout status deployment/my-app -n my-app-namespace --
timeout=2m
            kubectl get pods -n my-app-namespace -o wide
          '''
        }
      }
    }

    stage('Post-Deployment: Register Metrics & Dashboards') {
      steps {
        echo "Prometheus scrape annotations are in the k8s manifest. If using
Prometheus operator, apply ServiceMonitor."
        // optionally create ServiceMonitor:
        // kubectl apply -f k8s/servicemonitor.yaml
      }
    }

  } // stages

  post {
    success {
      echo "Pipeline succeeded. Image: ${DOCKER_REGISTRY}:${IMAGE_TAG}"
    }
    failure {
      echo "Pipeline failed. Check logs"
    }
    always {
      cleanWs()
    }
  }
}
```

Notes on `Jenkinsfile`:

- The pipeline uses labels (`maven`, `docker`, `kubectl`) so you can map stages to specific nodes/agents with required tools.
- `jfrog` CLI and `docker` must be installed on the respective Jenkins agents.

- Replace URLs, repo names, and credential IDs with your real values.
- You may integrate SonarQube via the Jenkins Sonar plugin (`withSonarQubeEnv`) if configured — I used a direct `mvn sonar:sonar` call with token.

---

2) `Dockerfile` (example for Spring Boot jar)

If you prefer to keep `Dockerfile` in repo instead of generating in Jenkins:

```
FROM eclipse-temurin:17-jre
ARG JAR_FILE=target/my-app.jar
COPY ${JAR_FILE} /app/app.jar
ENTRYPOINT ["java","-jar","/app/app.jar"]
EXPOSE 8080
```

---

3) Kubernetes Manifests (under `k8s/`)

`namespace.yaml`

```
apiVersion: v1
kind: Namespace
metadata:
  name: my-app-namespace
```

`deployment.yaml`

This manifest includes Prometheus scrape annotations (Pod annotations and Service) and resource requests/limits.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  namespace: my-app-namespace
  labels:
    app: my-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "8080"
        prometheus.io/path: "/actuator/prometheus"  # for Spring Boot Micrometer
endpoint
    spec:
      containers:
```

```
        - name: my-app
          image: DOCKER_IMAGE_PLACEHOLDER
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /actuator/health
              port: 8080
            initialDelaySeconds: 20
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /actuator/health
              port: 8080
            initialDelaySeconds: 60
            periodSeconds: 20
          resources:
            requests:
              memory: "512Mi"
              cpu: "250m"
            limits:
              memory: "1Gi"
              cpu: "1000m"
```

`service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: my-app
  namespace: my-app-namespace
  labels:
    app: my-app
spec:
  selector:
    app: my-app
  ports:
    - name: http
      port: 80
      targetPort: 8080
  type: ClusterIP
```

`servicemonitor.yaml` (if using Prometheus Operator)

Apply only if Prometheus Operator (kube-prometheus) is installed.

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: my-app-servicemonitor
  namespace: monitoring    # often the prometheus namespace or same as app
  labels:
    release: prometheus
spec:
  selector:
    matchLabels:
      app: my-app
  namespaceSelector:
    matchNames:
```

```
    - my-app-namespace
  endpoints:
    - port: http
      path: /actuator/prometheus
      interval: 15s
```

---

4) Application: expose metrics

- For Spring Boot, add Micrometer + Prometheus dependency and expose `/actuator/prometheus`. Example:
  - Add to `pom.xml`:

    ```
    <dependency>
      <groupId>io.micrometer</groupId>
      <artifactId>micrometer-registry-prometheus</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
    ```

  - In `application.properties`:

    ```
    management.endpoints.web.exposure.include=health,info,prometheus
    management.endpoint.prometheus.enabled=true
    management.endpoint.health.show-details=always
    management.server.port=8080
    ```

---

5) Prometheus & Grafana Setup (high level)

Option A — If you have Prometheus Operator:

- Install `kube-prometheus-stack` via Helm (includes Prometheus and Grafana).
- The `ServiceMonitor` will be picked up automatically if Prometheus is configured to watch that namespace.

Example Helm install:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
helm install prometheus prometheus-community/kube-prometheus-stack --namespace monitoring --create-namespace
```

Option B — If you run a standalone Prometheus:

- Configure `prometheus.yml` to include a `kubernetes_sd_configs` scrape job or enable `prometheus.io/*` pod annotations scraping by your Prometheus scrape config.

Example scrape config snippet:

```
- job_name: 'kubernetes-pods'
  kubernetes_sd_configs:
  - role: pod
  relabel_configs:
  - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
    action: keep
    regex: true
  - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
    action: replace
    target_label: __metrics_path__
    regex: (.+)
  - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_port]
    action: replace
    target_label: __address__
    regex: (.+)
    replacement: $1
```

Grafana:

- Import dashboards targeting the metrics your app exposes (Spring Boot Micrometer has common panels).
- If using `kube-prometheus-stack`, Grafana is installed and accessible as a service (check the helm chart notes).

---

6) JFrog Artifactory (upload specifics)

- Ensure `jfrog` CLI is installed on the Jenkins agent.
- Configure Artifactory connection: `jfrog rt c <config-name> --url=... --user=... --password=...`
- Upload via `jfrog rt u "target/*.jar" my-repo/path/ --build-name=... --build-number=...`
- Publish build info using `jfrog rt bce` and `jfrog rt bp`.

(These steps are shown in the Jenkinsfile already.)

---

7) Security & Best Practices (short list)

- Store tokens and credentials in **Jenkins Credentials** (never in plain text).
- Use **imagePullSecrets** if your registry is private (add to Deployment spec).
- Use **readiness/liveness/startup probes** correctly to avoid premature restarts.
- Tag container images with both build number and immutable tag (e.g., `1.2.3-${BUILD_NUMBER}`).

- Use **resource requests & limits** to avoid OOMKilled issues.
- Use **Helm** for more advanced deployment templating (easier to manage image tags and env vars).
- Consider **canary / blue-green** deployments for safer rollouts.

---

8) Optional: Helm Chart + Values (recommended next step)

If you want a cleaner deployment process, convert `k8s/*.yaml` into a Helm chart and add `values.yaml` with `image.tag` injected from Jenkins. Jenkins can run `helm upgrade --install my-app ./chart --set image.tag=$IMAGE_TAG`.

---

9) Example Troubleshooting tips

- If push to docker registry fails: check `docker.withRegistry` credential, registry endpoint, and network.
- If `kubectl apply` fails: check `KUBECONFIG` contents and namespace permissions (RBAC).
- If Sonar analysis fails: check `SONAR_TOKEN` and `sonar.host.url`.
- If Prometheus not scraping: ensure either `ServiceMonitor` matches labels and Prometheus Operator is listening in that namespace, OR Prometheus scrape config respects `prometheus.io/scrape` annotations.

---

10) Want this packaged?

I can:

- Generate a **ready-to-import Helm chart** for the app,
- Produce a **zip** with `Jenkinsfile`, `Dockerfile`, `k8s/` manifests, and a tiny `pom.xml` template,
- Or convert the document into a **PDF/Word** for handoff.